



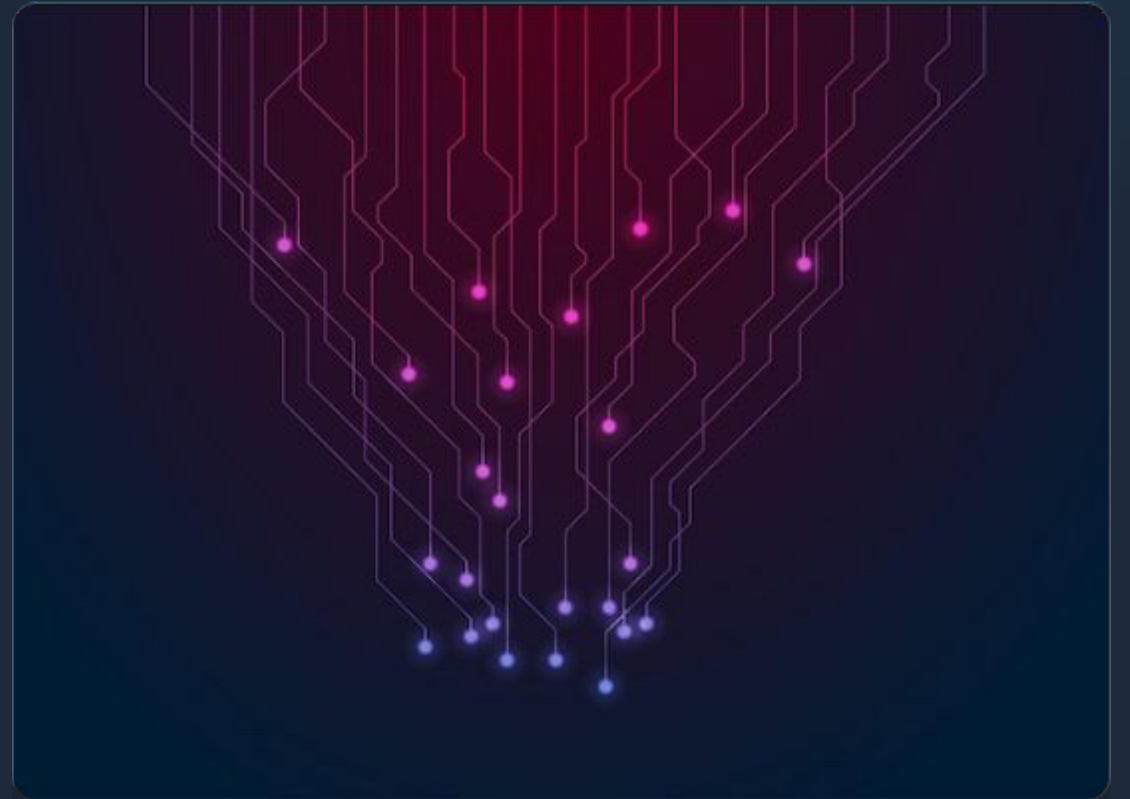
Présentation POC DataPress

Architecture Microservices & DevOps Modernes

Lucas - Novembre 2025

Introduction & Contexte

- **Contexte** : Démonstration pédagogique d'une architecture microservices complète.
- **Objectifs** : Illustrer les bonnes pratiques DevOps, la modularité et les stratégies de déploiement moderne.
- **Valeur Client** : Fournir une architecture maintenable, hautement évolutive et facilement déployable en production.



| Architecture Globale



Services

API (Flask +
Gunicorn)
Front (NGINX)
Auth (Flask)



Réseau

Communication
isolée via réseau
interne Docker
datapress-net



Ports

API: 5000
Front: 80
Auth: 5001



K8s

Deployment
Service
ConfigMap
Secret

Conteneurisation Docker

- **Dockerfile Unique** : Un fichier optimisé par service.
- **API Backend** : Multi-stage build, utilisateur non-root pour la sécurité, serveur WSGI Gunicorn.
- **Frontend** : Basé sur l'image légère nginx:alpine.
- **Optimisations** : Utilisation d'images slim et gestion du cache pip pour des builds rapides.



| Orchestration Locale (Compose)

Configuration

- **Fichier** : docker-compose.yml centralise la définition des services.
- **Dépendances** : Utilisation de depends_on pour l'ordre de démarrage.
- **Réseau** : Création automatique de datapress-net.

Exécution & Logs

- **Accessibilité** : Services exposés sur localhost.
- **Endpoints** : API répond sur / et /health.
- **Monitoring** : Logs agrégés accessibles via docker-compose logs.

Déploiement Kubernetes

- **Manifests** : Fichiers YAML standard situés dans k8s/.
- **Objets K8s** : Utilisation de Deployments pour le scaling et Services pour le networking.
- **Environnement** : Configuration via ConfigMaps et Secrets.
- **Commande** : Déploiement simple via `kubectl apply -f k8s/`.



| Sécurité & Hardening



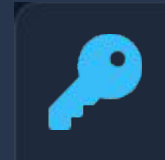
Non-Root

Exécution des conteneurs avec privilèges restreints.



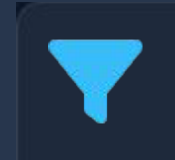
Isolation

Services confinés dans un réseau interne privé.



Secrets

Gestion des données sensibles via K8s Secrets.



Ports

Exposition limitée strictement au nécessaire.

| Supervision & Observabilité

État Actuel

- **Healthchecks** : Intégrés nativement dans Docker pour l'API.
- **Logs** : Centralisation simple via la sortie standard (stdout/stderr) des conteneurs.
- **Accès** : Visualisation rapide via CLI Docker.

Extensions Futures

- **Métriques** : Exportation de métriques applicatives personnalisées.
- **Stack** : Intégration de Prometheus pour la collecte et Grafana pour la visualisation.
- **Tracing** : Suivi des requêtes inter-services.

Commandes d'Exploitation

Démarrage :

`docker-compose up --build`

Arrêt & Nettoyage :

`docker-compose down --volumes`

Suivi des logs :

`docker-compose logs -f`

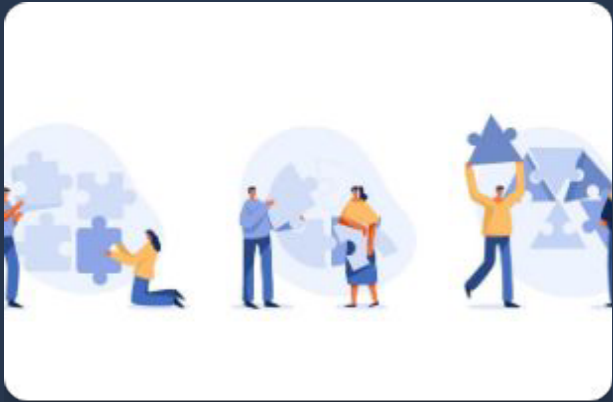
Tests de santé :

`curl http://localhost:5000/health`

`curl http://localhost:5001/health`

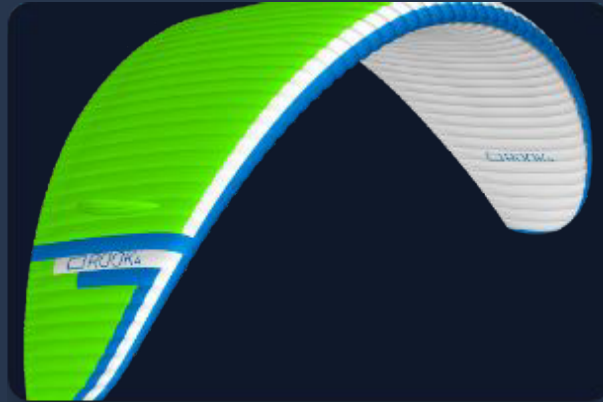


Synthèse des Bonnes Pratiques



Responsabilité Unique

Séparation stricte des services pour une maintenance aisée.



Optimisation

Images légères (Alpine/Slim) pour réduire la surface d'attaque.

 CI/CD devops infinite loop symbol

Automatisation

Architecture prête pour l'intégration continue (CI/CD).



Conclusion

Le POC démontre une architecture microservices **validée, sécurisée et supervisée.**

Une base solide prête pour l'échelle et la production.